

Kubernetes use cases on Azure for Cloud Solutions Architect

Kubernetes use cases on Azure for your Cloud Solutions Architect	1
1. Key Kubernetes Concepts (Azure Kubernetes Service - AKS)	2
2. Designing Effective & Efficient Cloud Solutions Using AKS	2
3. Sequence to Create Resources for Production Deployment	3
4. Best Practices for Production YAML Files	3
Folder Structure for Kubernetes Manifests:	3
Key YAML Components for Production	4
 End-to-End Production-Ready Kubernetes Solution on Azure: From Objectives to Outcome	 5
 1. Objectives & Requirements Phase	5
Key Objectives:	5
Business Requirements:	5
 2. Solution Design Phase	5
Architecture Overview:	5
 3. Design Layers and Their Implementation	6
✓ 1. Identity & Access Management (IAM) Layer:	6
✓ 2. Networking Layer:	7
✓ 3. Compute Layer:	8
✓ 4. Application Layer:	9
✓ 5. Data Layer:	10
✓ 6. Monitoring & Observability Layer:	10
✓ 7. CI/CD Layer:	11
✓ 8. Backup & Disaster Recovery:	11
 4. Testing Phase:	11
 5. Expected Outcomes:	12
 Setting up an Azure DevOps Pipeline for Kubernetes on Azure (AKS)	 12
Key Requirements:	12
1. Prerequisites Setup	13
1.1. Azure Service Connection in Azure DevOps	13
1.2. Kubernetes Cluster Setup	13
1.3. Azure Container Registry (ACR)	13
2. Create the Pipeline	13

2.1. Create a New Pipeline in Azure DevOps	13
3. Pipeline Configuration (YAML)	13
3.1. Define Variables	14
3.2. Build & Push Docker Image	14
3.3. Set Up Kubectl for AKS	15
3.4. Deploy to AKS Using Helm	15
3.5. Monitoring & Logging Integration	16
3.6. Clean-Up and Deployment Validation	16
4. Pipeline Overview	17
5. Final Steps	19

1. Key Kubernetes Concepts (Azure Kubernetes Service - AKS)

- Cluster Architecture: Master Node (Control Plane) & Worker Nodes
 - Pods: Smallest deployable units, running one or more containers
 - Namespaces: Logical isolation for different environments (dev, staging, prod)
 - Deployments: Declarative updates for Pods
 - Services: ClusterIP, NodePort, LoadBalancer, ExternalName
 - Ingress: Manages external access to services via URLs
 - ConfigMaps & Secrets: Store configuration data and sensitive information
 - Persistent Volumes (PV) & Persistent Volume Claims (PVC): Storage management
 - Helm: Package manager for Kubernetes applications
 - RBAC (Role-Based Access Control): Manage access to cluster resources
 - Networking: CNI, Network Policies, Azure CNI, and Kubelet
-

2. Designing Effective & Efficient Cloud Solutions Using AKS

- Architecture Patterns:
 - Microservices Architecture
 - Multi-Cluster Deployments for High Availability
 - Hybrid Cloud Deployments (using Azure Arc)
- Scaling: Horizontal Pod Autoscaler (HPA) & Vertical Pod Autoscaler (VPA)
- Monitoring & Logging: Azure Monitor, Azure Log Analytics, Prometheus, Grafana
- Security: Azure AD Integration, Secrets Management, Network Policies, Azure Key Vault
- Disaster Recovery: Cross-region AKS replication, Backup strategies using Velero

Example Design Pattern:

- Web Layer: Ingress Controller, Nginx Controller
 - App Layer: Microservices hosted on Pods, managed with Deployments
 - DB Layer: Azure Managed Databases (e.g., Cosmos DB, Azure SQL)
 - Identity & Access Management: Azure AD Integration
 - Common Services: Azure Monitor, Azure Key Vault, Backup via Velero
-

3. Sequence to Create Resources for Production Deployment

1. Resource Group Creation: Create a logical grouping for AKS resources.
 2. Identity Management: Configure Azure AD integration for RBAC.
 3. AKS Cluster Creation: Set up AKS with proper node pools and autoscaling configurations.
 4. Networking Setup: Define VNet, Subnets, and configure Azure CNI/Kubenet.
 5. Storage Configuration: Create Persistent Volumes and Claims.
 6. Application Deployment: Deploy application workloads using YAML manifests.
 7. Ingress Controller: Set up Nginx or Application Gateway Ingress.
 8. Monitoring & Logging: Integrate Azure Monitor, Log Analytics.
 9. Backup & Disaster Recovery: Implement backup strategies using Velero.
 10. CI/CD Pipeline: Automate deployments using Azure DevOps or GitHub Actions.
-

4. Best Practices for Production YAML Files

Folder Structure for Kubernetes Manifests:

```
k8s/
├── base/                # Base templates for Kubernetes resources
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── ingress.yaml
│   ├── configmap.yaml
│   └── secret.yaml
├── overlays/
│   ├── dev/
│   │   ├── deployment.yaml
│   │   └── kustomization.yaml
│   └── prod/
│       ├── deployment.yaml
│       └── kustomization.yaml
```

Key YAML Components for Production:

1. Deployment YAML: Define Pod specs, resource limits, and auto-scaling.
2. Service YAML: Use LoadBalancer or ClusterIP for communication.
3. Ingress YAML: Define URL routing and TLS configurations.
4. ConfigMap & Secrets YAML: Store configurations securely.
5. PersistentVolume & PersistentVolumeClaim YAML: For storage requirements.
6. RBAC YAML: Grant necessary permissions to service accounts.
7. Monitoring YAML: Integration with Prometheus or Azure Monitor.

Let me know if you need deeper insights into any of these sections, or if you'd like practice questions specific to Kubernetes and Azure scenarios! 🚀

End-to-End Production-Ready Kubernetes Solution on Azure: From Objectives to Outcome

This guide breaks down an Azure Kubernetes Service (AKS) production solution into clear phases, covering objectives, design, architecture, implementation, and expected outcomes across each layer.

1. Objectives & Requirements Phase

Key Objectives:

- Build a scalable, reliable, and secure cloud-native application on Azure Kubernetes Service (AKS).
- Ensure high availability (HA), disaster recovery (DR), and fault tolerance.
- Implement CI/CD pipelines for efficient and automated deployments.
- Enable monitoring, logging, and tracing for visibility and observability.
- Ensure data security, compliance, and role-based access controls (RBAC).

Business Requirements:

- 99.9% uptime SLA
 - Multi-region failover support
 - Microservices architecture
 - Cost optimization
-

2. Solution Design Phase

Architecture Overview:

The architecture consists of multiple layers:

1. Identity & Access Management Layer: Azure AD, RBAC
2. Networking Layer: Azure Virtual Network (VNet), Subnets, Private Endpoints, Ingress Controller

3. Compute Layer: AKS Node Pools (System and User Nodes)
 4. Application Layer: Microservices on AKS, API Gateway
 5. Data Layer: Azure SQL Database, Azure Cosmos DB
 6. Storage Layer: Persistent Volumes (PV), Persistent Volume Claims (PVC)
 7. Monitoring & Observability Layer: Azure Monitor, Log Analytics, Prometheus, Grafana
 8. CI/CD Layer: Azure DevOps, GitHub Actions
 9. Backup & Disaster Recovery Layer: Velero, Azure Backup
-

3. Design Layers and Their Implementation

1. Identity & Access Management (IAM) Layer:

Objective: Secure access to AKS and resources.

- Azure Active Directory (Azure AD): Centralized identity management.
- Role-Based Access Control (RBAC): Fine-grained access permissions.
- Azure Key Vault: Manage secrets, certificates, and keys securely.
- Service Principal/Managed Identity: Secure cluster interaction with Azure services.

YAML Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: read-secrets-binding
  namespace: production
subjects:
- kind: User
  name: user@example.com
  apiGroup: rbac.authorization.k8s.io
roleRef:
  kind: Role
  name: read-secrets
```

`apiGroup: rbac.authorization.k8s.io`

✓ 2. Networking Layer:

Objective: Ensure secure and optimized network communication.

- Azure Virtual Network (VNet): Isolate Kubernetes clusters in private networks.
- Azure Private Link: Secure connections to Azure services.
- Ingress Controller: Nginx Ingress for routing traffic.
- Network Policies: Control traffic between pods.

YAML Example:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-internal
  namespace: production
spec:
  podSelector: {}
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector:
          matchLabels:
            role: backend
```

✓ 3. Compute Layer:

Objective: Optimize resource utilization and reliability.

- Node Pools: Separate system and application workloads.
- Auto-Scaling: Horizontal Pod Autoscaler (HPA) and Cluster Autoscaler.
- Spot VM Integration: Cost optimization for non-critical workloads.

YAML Example (HPA):

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: backend-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: backend
  minReplicas: 2
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 75
```

✓ 4. Application Layer:

Objective: Host microservices architecture efficiently.

- Deployments: Define containerized app behavior.
- Services: ClusterIP, LoadBalancer.
- ConfigMaps & Secrets: Manage app configurations securely.

YAML Example (Deployment):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  namespace: production
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: backend:v1
          ports:
            - containerPort: 80
          envFrom:
            - configMapRef:
                name: backend-config
```

✓ 5. Data Layer:

Objective: Reliable and scalable data management.

- Azure Cosmos DB: NoSQL storage.
- Azure SQL DB: Managed relational database.
- Persistent Volumes (PV): For stateful services.

YAML Example (PVC):

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: backend-pvc
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 10Gi
```

✓ 6. Monitoring & Observability Layer:

Objective: Ensure visibility into performance and security.

- Azure Monitor: Monitor health, availability, and performance.
- Prometheus & Grafana: Real-time metrics and dashboards.

YAML Example (Prometheus Config):

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
```

```
scrape_configs:
  - job_name: 'kubernetes'
    kubernetes_sd_configs:
      - role: node
```

✓ 7. CI/CD Layer:

Objective: Streamline deployment workflows.

- Azure DevOps Pipelines: Automate build and deployment.
 - Helm: Package Kubernetes apps for deployment.
-

✓ 8. Backup & Disaster Recovery:

Objective: Minimize data loss and downtime.

- Velero: Backup Kubernetes cluster and restore.
 - Azure Backup Service: Manage storage backups.
-



4. Testing Phase:

- Unit Testing: Validate each microservice.
 - Integration Testing: Ensure smooth service interactions.
 - End-to-End Testing: Validate entire application workflow.
-

5. Expected Outcomes:

- Scalable & Secure Architecture: Support increasing workloads.
- Automated Deployments: Fast, reliable CI/CD pipelines.
- Improved Observability: Real-time insights via monitoring tools.
- Cost Efficiency: Optimized resource usage.
- Compliance & Security: Azure AD, RBAC, and Secrets Management.

Setting up an Azure DevOps Pipeline for Kubernetes on Azure (AKS)

Here's a step-by-step breakdown of how to set up an Azure DevOps Pipeline for deploying applications to Azure Kubernetes Service (AKS). This pipeline covers everything from building your application to deploying it securely with monitoring.

Key Requirements:

- Azure DevOps account.
 - Azure Kubernetes Service (AKS) cluster.
 - Azure DevOps service connection to your Azure account.
 - Helm for Kubernetes package management.
-

1. Prerequisites Setup

Before setting up the pipeline, ensure the following are ready:

1.1. Azure Service Connection in Azure DevOps

- In Azure DevOps, go to Project Settings > Service connections.
- Create a new Azure Resource Manager service connection (this will allow your pipeline to interact with Azure services).
- This connection will be used for interacting with AKS, Azure Container Registry (ACR), and other Azure resources.

1.2. Kubernetes Cluster Setup

- Ensure your AKS cluster is up and running.

- Ensure you have configured kubectl and Helm for Azure DevOps agents.

1.3. Azure Container Registry (ACR)

- Ensure you have an Azure Container Registry to store Docker images.
 - Push your Docker image to ACR as part of the pipeline.
-

2. Create the Pipeline

2.1. Create a New Pipeline in Azure DevOps

1. Go to your Azure DevOps project.
 2. Under Pipelines, click New Pipeline.
 3. Select your repository (e.g., GitHub, Azure Repos Git, or GitLab).
 4. Choose YAML pipeline (instead of classic pipeline).
-

3. Pipeline Configuration (YAML)

Here's a breakdown of the YAML pipeline file to automate the build, push, and deployment process:

3.1. Define Variables

Add variables to store necessary information like Azure subscription, AKS cluster name, and container registry.

variables:

```
azureSubscription: 'YourAzureSubscription' # Azure
```

Service Connection

```
aksClusterName: 'YourAKSClusterName'
```

```
aksResourceGroup: 'YourResourceGroupName'
```

```
acrName: 'youracrname.azurecr.io' # Azure
```

Container Registry

```
imageName: 'yourapp-image'
```

```
appName: 'yourapp'
kubernetesNamespace: 'default'
```

3.2. Build & Push Docker Image

The first phase is building the Docker image from your application and pushing it to Azure Container Registry.

```
stages:
- stage: Build
  jobs:
  - job: BuildAndPush
    pool:
      vmImage: 'ubuntu-latest'
    steps:
    - task: Docker@2
      inputs:
        containerRegistry: $(azureSubscription)
        repository: $(acrName)/$(imageName)
        command: 'buildAndPush'
        Dockerfile: '$(Build.SourcesDirectory)/Dockerfile'
        tags: |
          $(Build.BuildId)
```

3.3. Set Up Kubectl for AKS

In the next step, set up the kubectl tool to interact with AKS. This uses the Azure service connection to get access to your AKS cluster.

```
- task: AzureCLI@2
  inputs:
    azureSubscription: $(azureSubscription)
    scriptType: 'pscore'
    scriptLocation: 'inlineScript'
    inlineScript: |
```

```
    az aks get-credentials --resource-group  
$(aksResourceGroup) --name $(aksClusterName)  
    displayName: 'Get AKS Credentials'
```

3.4. Deploy to AKS Using Helm

You'll now use Helm to deploy the Kubernetes charts (your application) to AKS.

```
- task: HelmInstaller@1  
  inputs:  
    helmVersionToInstall: 'latest'  
  
- task: HelmDeploy@0  
  inputs:  
    connectionType: 'Azure Resource Manager'  
    azureSubscription: $(azureSubscription)  
    azureResourceGroup: $(aksResourceGroup)  
    kubernetesCluster: $(aksClusterName)  
    namespace: $(kubernetesNamespace)  
    chartType: 'FilePath'  
    chartPath: '$(Build.SourcesDirectory)/helm/yourapp-chart'  
    releaseName: $(appName)  
    overrideValues: |  
      image.repository=$(acrName)/$(imageName)  
      image.tag=$(Build.BuildId)  
    install: true  
    waitForDeployment: true
```

3.5. Monitoring & Logging Integration

Set up monitoring and logging through Azure Monitor and Log Analytics. This can be done by configuring Helm to install monitoring tools like Prometheus and Grafana for real-time metrics collection.

```
- task: HelmDeploy@0
```

```
inputs:
  connectionType: 'Azure Resource Manager'
  azureSubscription: $(azureSubscription)
  azureResourceGroup: $(aksResourceGroup)
  kubernetesCluster: $(aksClusterName)
  namespace: 'monitoring'
  chartType: 'Helm Chart'
  chartPath: 'stable/prometheus'
  releaseName: 'prometheus'
  install: true
  waitForDeployment: true
```

3.6. Clean-Up and Deployment Validation

You can add a post-deployment validation step to ensure everything is running smoothly.

```
- task: Kubernetes@1
  inputs:
    connectionType: 'Azure Resource Manager'
    azureSubscription: $(azureSubscription)
    kubernetesCluster: $(aksClusterName)
    command: 'get'
    arguments: 'pods --namespace $(kubernetesNamespace)'
    displayName: 'Check Pods Status'
```

4. Pipeline Overview

Here is the complete breakdown of the Azure DevOps pipeline:

```
trigger:
- main # Change the branch as per your source branch for deployment
```

variables:

```
azureSubscription: 'YourAzureSubscription'
aksClusterName: 'YourAKSClusterName'
aksResourceGroup: 'YourResourceGroupName'
acrName: 'youracrname.azurecr.io'
imageName: 'yourapp-image'
appName: 'yourapp'
kubernetesNamespace: 'default'
```

stages:

- stage: Build

jobs:

- job: BuildAndPush

pool:

vmImage: 'ubuntu-latest'

steps:

- task: Docker@2

inputs:

containerRegistry: \$(azureSubscription)

repository: \$(acrName)/\$(imageName)

command: 'buildAndPush'

Dockerfile: '\$(Build.SourcesDirectory)/Dockerfile'

tags: |

\$(Build.BuildId)

- stage: Deploy

jobs:

- job: DeployToAKS

pool:

vmImage: 'ubuntu-latest'

steps:

- task: AzureCLI@2

inputs:

azureSubscription: \$(azureSubscription)

```
    scriptType: 'pscore'
    scriptLocation: 'inlineScript'
    inlineScript: |
        az aks get-credentials --resource-group
$(aksResourceGroup) --name $(aksClusterName)
        displayName: 'Get AKS Credentials'

- task: HelmInstaller@1
  inputs:
    helmVersionToInstall: 'latest'

- task: HelmDeploy@0
  inputs:
    connectionType: 'Azure Resource Manager'
    azureSubscription: $(azureSubscription)
    azureResourceGroup: $(aksResourceGroup)
    kubernetesCluster: $(aksClusterName)
    namespace: $(kubernetesNamespace)
    chartType: 'FilePath'
    chartPath:
'$(Build.SourcesDirectory)/helm/yourapp-chart'
    releaseName: $(appName)
    overrideValues: |
        image.repository=$(acrName)/$(imageName)
        image.tag=$(Build.BuildId)
    install: true
    waitForDeployment: true

- task: Kubernetes@1
  inputs:
    connectionType: 'Azure Resource Manager'
    azureSubscription: $(azureSubscription)
    kubernetesCluster: $(aksClusterName)
    command: 'get'
    arguments: 'pods --namespace $(kubernetesNamespace)'
```

`displayName: 'Check Pods Status'`

5. Final Steps

1. **Save and Run the Pipeline:** Once you've completed the YAML, commit and push the pipeline file to your repository. This will trigger the pipeline to run on each push to your main branch.
 2. **Monitor:** Ensure all steps run successfully and monitor the deployment, logs, and metrics.
-

Let me know if you need further guidance on a specific section or troubleshooting tips! 🚀

Hope you find this document helpful!

For more such content you can check : <https://techyoutube.com/>

Telegram: <https://t.me/LearnDevOpsForFree>

Twitter: <https://twitter.com/techyoutbe>

Youtube: <https://www.youtube.com/@T3Ptech>